

Deadlock Detection Tool

James Widner
KSU-ID: 001121770

May 4, 2026

Abstract

Deadlock is a major problem in operating systems because multiple processes can hold resources while waiting indefinitely for resources held by other processes. This project implements a Go-based deadlock detection simulator that reads a resource allocation state, calculates available resources, builds a wait-for graph, detects cycles, performs a safe-state check, and recommends recovery or prevention actions. The project shows that deadlock detection is practical when the operating system has a known finite state. At the same time, it explains an important theoretical limitation: perfectly predicting whether every arbitrary program will eventually deadlock is not generally solvable, because such prediction requires reasoning about all possible future execution paths, inputs, schedules, and resource requests. This limitation is related to undecidability ideas behind the Halting Problem.

1 Introduction

Operating systems allow many programs to run at the same time. This concurrency improves performance and responsiveness, but it also creates competition for limited resources. Processes may need files, printers, memory regions, database locks, network connections, or other exclusive resources. If a process holds one resource while waiting for another, and another process does the same thing in a circular chain, the system can enter a deadlock.

Deadlock matters because a deadlocked process cannot make progress by itself. In a real system this can cause an application to freeze, a server request to hang, or an entire subsystem to become unavailable. Because operating systems must manage shared resources, they need methods for detecting, avoiding, preventing, or recovering from deadlock.

The purpose of this project is to build a small but clear deadlock detection tool. The tool does not attempt to control the real kernel. Instead, it simulates the resource allocation state of an operating system. It uses a graph-based model to explain which processes are waiting for which other processes. If the graph contains a cycle, the tool reports deadlock and suggests ways to break it.

2 Background Information and Motivation

A deadlock can occur when four necessary conditions hold at the same time. The first condition is mutual exclusion, meaning at least one resource cannot be shared freely. The second is hold and wait, meaning a process holds one resource while waiting for another. The third is no preemption, meaning the system does not forcibly take the resource away. The fourth is circular wait, meaning there is a cycle of processes where each process waits for a resource held by the next process in the cycle.

Operating systems textbooks usually describe several approaches for handling deadlock. Deadlock prevention tries to make at least one of the four necessary conditions impossible. For example, a system can require all processes to request resources in the same global order, which prevents circular wait. Deadlock avoidance checks whether granting a resource request would keep the system in a safe state. Banker's Algorithm is a classic example of this approach. Deadlock detection allows the system to enter a deadlocked state, but periodically checks for it and then recovers by aborting, rolling back, or preempting a process.

This project focuses on detection. Detection is useful because it is concrete and easy to demonstrate. Given a fixed set of processes, resources, allocations, and requests, the current state can be modeled as a graph. A graph cycle can be found with a standard depth-first search. This makes the current deadlock detection problem solvable for finite states.

However, this does not mean all future deadlocks can be predicted perfectly. A general deadlock predictor for arbitrary programs would need to know whether some future execution path will eventually reach a deadlocked state. That depends on program behavior, input, scheduling, branches, loops, and timing. This makes the broad prediction problem similar in spirit to the Halting Problem: it asks for a perfect answer about all possible behaviors of arbitrary programs. The simulator therefore separates what operating systems can practically solve from what is theoretically impossible in full generality.

3 Problem Definition

The project solves the following practical problem:

Given a finite snapshot of processes, resources, current allocations, and current outstanding requests, determine whether the system is currently deadlocked. If deadlock exists, identify the cycle of processes and recommend recovery or prevention actions.

The input is not a program source file and the tool does not predict every possible future schedule. Instead, it analyzes a resource allocation snapshot. This keeps the problem well-defined and solvable.

4 Proposed Solution

The proposed solution is to represent the system as a wait-for graph. In a wait-for graph, each node is a process. If process P_1 is waiting for a resource currently held by process P_2 , the graph contains a directed edge from P_1 to P_2 . A cycle means that each process in the cycle is waiting for another process in the same cycle. Since none of those processes can continue without outside intervention, the cycle represents a deadlock.

For example, suppose the system has three processes:

- P_1 holds the printer and requests the scanner.
- P_2 holds the scanner and requests the database.
- P_3 holds the database and requests the printer.

The wait-for graph is:

$$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_1$$

This cycle proves that the system is deadlocked.

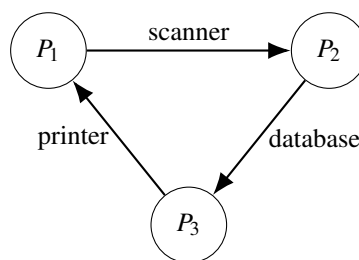


Figure 1: Wait-for graph for the sample deadlock. Each arrow points from a waiting process to the process holding the needed resource.

5 Methods

The implementation follows six main steps.

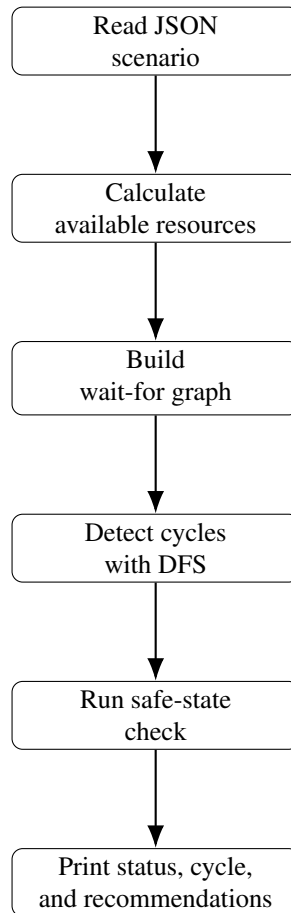


Figure 2: Analysis workflow used by the deadlock detection tool.

5.1 Input Parsing

The program reads a JSON file with three maps: total resources, current allocations, and current requests. Allocation and request keys use the format `process:resource`. This format keeps the input readable while allowing the program to model several processes and resources.

Listing 1: Example deadlock scenario

```
{
  "resources": {
    "printer": 1,
    "scanner": 1,
    "database": 1
  },
  "allocations": {
    "P1:printer": 1,
    "P2:scanner": 1,
    "P3:database": 1
  },
  "requests": {
    "P1:scanner": 1,
```

```
"P2:database": 1,  
"P3:printer": 1  
}  
}
```

5.2 Validation

The tool validates the scenario before detection. It checks that resource totals are not negative, allocation and request amounts are not negative, all referenced resources exist, and no resource is over-allocated. This avoids reporting graph results for an invalid operating-system state.

5.3 Available Resource Calculation

For each resource, the tool subtracts allocated instances from the total number of instances:

$$Available(R) = Total(R) - \sum Allocated(P, R)$$

This determines whether a pending request can be satisfied immediately or whether it must wait for another process to release the resource.

5.4 Wait-For Graph Construction

For every request that cannot be satisfied with currently available resources, the tool finds processes that hold the requested resource. If process P_i requests resource R , and process P_j currently holds R , the tool adds the edge:

$$P_i \rightarrow P_j$$

Multiple resources can create multiple wait relationships, but duplicate edges are removed for clarity.

5.5 Cycle Detection

The program uses depth-first search to find cycles in the wait-for graph. During traversal, each process is marked as unvisited, currently visiting, or completed. If the search reaches a process that is already in the current recursion stack, a cycle has been found. The cycle is then reported as the deadlocked group of processes.

5.6 Safe-State Check

The tool also performs a safe-state check inspired by Banker's Algorithm. It repeatedly looks for a process whose current outstanding requests can be satisfied by the available resources. If such a process can finish, the tool releases its allocated resources into the available pool and continues. If all processes can finish, the state is safe. If some processes cannot finish, the state is unsafe.

This safe-state result is not the same as the cycle result, but it gives additional insight. A current deadlock is unsafe, while some unsafe states may not yet contain a concrete wait-for cycle.

6 Implementation and Experimental Results

The project is implemented in Go. The command-line entry point is located in `cmd/deadlock`. The core detection logic is located in `internal/deadlock`. The examples are located in `examples`.

The project can be run with:

```
go run ./cmd/deadlock -file examples/deadlock.json
```

The test suite can be run with:

```
go test ./...
```

6.1 Experiment 1: Deadlocked System

The first experiment uses three resources and three processes. Each process holds one resource and requests the next resource in a cycle. The output is:

```
jameswidner@Jamess-MacBook-Pro deadlock-detection-tool % go run ./cmd/deadlock -file examples/deadlock.json

Deadlock Detection Tool
=====

Available resources:
  database: 0
  printer: 0
  scanner: 0

Wait-for graph:
  P1 -> P2
  P2 -> P3
  P3 -> P1

Status: DEADLOCK DETECTED
  Cycle 1: P1 -> P2 -> P3 -> P1
Safe-state check: unsafe
Unable to safely finish: P1, P2, P3

Recommended actions:
- Abort or roll back one process in a detected cycle to release its resources.
- Require all processes to request shared resources in the same global order.
- Use timeouts for long waits and retry after releasing partial allocations.
- Before granting a request, run a safe-state check similar to Banker's Algorithm.
- Detected cycle: P1 -> P2 -> P3 -> P1. Breaking any edge in this cycle removes the immediate deadlock.
- Blocked processes: P1, P2, P3. Inspect their outgoing wait-for edges first.
```

Figure 3: Terminal screenshot from running `go run ./cmd/deadlock -file examples/deadlock.json`. The tool detects the cycle $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_1$ and reports an unsafe state.

This confirms that the tool correctly identifies the circular wait condition. Since all resources have zero availability and every process is waiting on another process in the same cycle, no process can finish without intervention.

6.2 Experiment 2: Safe System

The second experiment includes enough available resources for the system to make progress. The output is:

This shows that waiting does not automatically mean deadlock. Process P_1 is waiting for P_2 , but P_2 can still finish, release resources, and allow P_1 to continue. The safe sequence demonstrates one possible order in which the system can complete all processes.

6.3 Testing

The project includes automated unit tests. The tests check that the tool detects a two-process cycle, finds a safe sequence in a safe scenario, and rejects invalid over-allocation. These tests help verify the main correctness properties of the simulator.

7 Discussion

The graph-based design is useful because it explains the reason for the deadlock. A simple status message such as “deadlocked” is not enough for recovery. The operating system or administrator needs to know which processes are involved. The wait-for graph gives that explanation directly.

```
jameswidner@Jamess-MacBook-Pro deadlock-detection-tool % go run ./cmd/deadlock -file examples/safe.json

Deadlock Detection Tool
=====

Available resources:
  database: 1
  printer: 1
  scanner: 0

Wait-for graph:
  P1 -> P2

Status: no current deadlock detected
Safe-state check: safe
Safe sequence: P2, P3, P1

Recommended actions:
  - No deadlock was found. Continue monitoring resource requests before granting new allocations.
jameswidner@Jamess-MacBook-Pro deadlock-detection-tool %
```

Figure 4: Terminal screenshot from running `go run ./cmd/deadlock -file examples/safe.json`. The tool reports no current deadlock and gives the safe sequence P2, P3, P1.

The recommendations produced by the tool are also connected to operating-system concepts. Aborting or rolling back one process in a cycle is a recovery method. Requiring a global resource order is a prevention method. Checking safe states before granting requests is an avoidance method. Timeouts are a practical engineering technique that can reduce indefinite waiting.

The project also supports a theoretical discussion. Deadlock detection for the current snapshot is decidable because the graph is finite. Cycle detection will always terminate. In contrast, predicting whether any arbitrary program will ever reach a deadlock is not completely solvable in general. A program may depend on unknown input, non-deterministic scheduling, or loops that may or may not terminate. A perfect future predictor would need to solve questions about arbitrary program behavior, which is why the broader prediction problem is connected to undecidability.

8 Limitations

The main limitation is that the tool analyzes a snapshot rather than a live operating system. It does not inspect actual kernel locks, actual file descriptors, or real process scheduling. The input scenario must be written manually in JSON.

Another limitation is that the safe-state check is simplified. Full Banker's Algorithm usually requires maximum claims for each process. This simulator uses current outstanding requests and current allocations, which is enough for demonstration but does not cover every full avoidance scenario.

The tool also does not decide which process should be aborted. It recommends breaking a cycle, but a real operating system would also consider priority, amount of work lost, user impact, and whether a process can safely roll back.

9 Conclusion

This project demonstrates a practical deadlock detection method for operating systems. By modeling processes and resources as a wait-for graph, the tool can detect cycles and identify the exact processes involved in a current deadlock. It also performs a safe-state check and suggests recovery, prevention, and avoidance strategies.

The project shows an important distinction: current deadlock detection for a finite state is solvable, but perfect

future deadlock prediction for arbitrary programs is not generally solvable. This distinction makes deadlock a useful topic for both practical operating-system design and theoretical computer science.

References

- [1] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts*. Wiley.
- [2] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson.
- [3] Edsger W. Dijkstra. Cooperating Sequential Processes. In *Programming Languages*, Academic Press, 1968.
- [4] Alan M. Turing. On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 1936.